Experiment 7

Hadoop Installation and MapReduce WordCount

Step-by-step practice guide for Ubuntu

Aim	Install Hadoop and run a sample MapReduce WordCount program in Ubuntu.
Hadoop version in record	Hadoop 3.3.1
Final proof	HDFS and YARN daemons are running, WordCount completes, and the output file can be displayed.

> Before starting: Open Ubuntu Terminal. Commands beginning with `sudo` may ask for your Ubuntu password.

Part A - Install Java and Hadoop

Step 1: Update Ubuntu

```
sudo apt update
```

Wait until the package lists finish updating.

Step 2: Install Java (JDK)

```
sudo apt install openjdk-8-jdk -y
java -version
```

> Practice note: Your record says to install JDK 8, but a later `hadoop-env.sh` example uses the Java 11 path. Use the JAVA_HOME path that matches the Java version you actually installed.

If you installed JDK 8, a common path is:

```
/usr/lib/jvm/java-8-openjdk-amd64
```

If Java 11 is installed and you choose to use it, the record shows:

```
/usr/lib/jvm/java-11-openjdk-amd64
```

Step 3: Download and extract Hadoop 3.3.1

Download the Hadoop 3.3.1 tar.gz file from Apache. Place it in your Downloads folder, then run:

```
cd ~/Downloads
tar -xzf hadoop-3.3.1.tar.gz
mv hadoop-3.3.1 ~/
```

Hadoop will now be in `~/hadoop-3.3.1`.

Step 4: Add Hadoop environment variables in .bashrc

```
nano ~/.bashrc
```

Go to the bottom of the file and add:

```
export HADOOP_HOME=$HOME/hadoop-3.3.1
export HADOOP_INSTALL=$HADOOP_HOME
export HADOOP_MAPRED_HOME=$HADOOP_HOME
export HADOOP_COMMON_HOME=$HADOOP_HOME
export HADOOP_HDFS_HOME=$HADOOP_HOME
export YARN_HOME=$HADOOP_HOME
export HADOOP_COMMON_LIB_NATIVE_DIR=$HADOOP_HOME/lib/native
export PATH=$PATH:$HADOOP_HOME/sbin:$HADOOP_HOME/bin
export HADOOP_OPTS="-Djava.library.path=$HADOOP_HOME/lib/native"
```

Save in nano: Ctrl+O -> Enter -> Ctrl+X. Then reload the file:

```
source ~/.bashrc
hadoop version
```

Step 5: Set JAVA_HOME in hadoop-env.sh

```
nano $HADOOP_HOME/etc/hadoop/hadoop-env.sh
```

Add or update the JAVA_HOME line. For JDK 8:

```
export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
```

If you installed Java 11 instead, use the Java 11 path shown in your record. Save and exit nano.

Part B - Configure Hadoop

Step 6: Configure core-site.xml

```
nano $HADOOP_HOME/etc/hadoop/core-site.xml
```

Replace the configuration section with:

```
<configuration>
 <property>
 <name>fs.defaultFS</name>
 <value>hdfs://localhost:9000</value>
 </property>
</configuration>
```

Step 7: Create HDFS storage folders and configure hdfs-site.xml

The record stores NameNode and DataNode data under `/usr/local/hadoop/hadoop_data/hdfs`. Create those folders first:

```
sudo mkdir -p /usr/local/hadoop/hadoop_data/hdfs/namenode
sudo mkdir -p /usr/local/hadoop/hadoop_data/hdfs/datanode
sudo chown -R $USER:$USER /usr/local/hadoop
```

```
nano $HADOOP_HOME/etc/hadoop/hdfs-site.xml
```

```
<configuration>
 <property>
 <name>dfs.replication</name>
 <value>1</value>
 </property>
 <property>
 <name>dfs.namenode.name.dir</name>
 <value>file:/usr/local/hadoop/hadoop_data/hdfs/namenode</value>
 </property>
 <property>
 <name>dfs.datanode.data.dir</name>
 <value>file:/usr/local/hadoop/hadoop_data/hdfs/datanode</value>
 </property>
</configuration>
```

Step 8: Configure mapred-site.xml

```
nano $HADOOP_HOME/etc/hadoop/mapred-site.xml
```

```
<configuration>
 <property>
 <name>mapreduce.framework.name</name>
 <value>yarn</value>
 </property>
</configuration>
```

Step 9: Configure yarn-site.xml

```
nano $HADOOP_HOME/etc/hadoop/yarn-site.xml
```

```
<configuration>
 <property>
 <name>yarn.nodemanager.aux-services</name>
 <value>mapreduce_shuffle</value>
 </property>
</configuration>
```

At this point, the four main configuration files from the record are complete: core-site.xml, hdfs-site.xml, mapred-site.xml, yarn-site.xml.

Part C - Start Hadoop

Step 10: Format the NameNode - first setup only

```
hdfs namenode -format
```

Important: Format the NameNode only when setting up a fresh HDFS. Re-formatting later can remove access to existing HDFS metadata.

Step 11: Start HDFS

```
start-dfs.sh
```

You should see messages similar to `Starting namenodes on [localhost]` and `Starting datanodes`.

Step 12: Start YARN

```
start-yarn.sh
```

You should see ResourceManager and NodeManager starting.

Step 13: Check daemon status using jps

```
jps
```

The record shows these important processes running: NameNode, DataNode, SecondaryNameNode, ResourceManager, NodeManager, plus Jps itself.

If start-dfs.sh asks for SSH/password or fails to connect to localhost: Hadoop pseudo-distributed mode may need localhost SSH configured. This is a troubleshooting note; it is not listed in your record.

Part D - Run the MapReduce WordCount program

Step 14: Create a local input text file

```
cd ~
echo "hello hadoop hadoop is powerful hello world" > file.txt
cat file.txt
```

Step 15: Create an input directory in HDFS and upload the file

```
hdfs dfs -mkdir -p /input
hdfs dfs -put file.txt /input
hdfs dfs -ls /input
```

If `file.txt` already exists in HDFS, remove it first with `hdfs dfs -rm /input/file.txt` and upload again.

Step 16: Run the Hadoop WordCount example

The record uses Hadoop's example MapReduce JAR. This practice command uses a wildcard so it automatically matches the installed Hadoop 3.3.x example JAR:

```
hdfs dfs -rm -r -f /output
hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-*.jar wordcount
/input /output
```

The first command removes an old `/output` directory if one exists. Hadoop WordCount will fail if the output directory already exists.

Step 17: View the output

```
hdfs dfs -ls /output
hdfs dfs -cat /output/part-r-00000
```

You should see each word followed by its count. The output screenshot in the record shows values such as Hello 2, is 1, and powerful. 1.

Quick Full Practice Sequence

Use this page when you already understand the setup and want to rehearse the experiment quickly.

#	Task	Command / Action
1	Update	sudo apt update
2	Install Java	sudo apt install openjdk-8-jdk -y
3	Extract Hadoop	tar -xzf hadoop-3.3.1.tar.gz && mv hadoop-3.3.1 ~/
4	Set variables	Edit ~/.bashrc, then: source ~/.bashrc
5	Set JAVA_HOME	Edit $HADOOP_HOME/etc/hadoop/hadoop-env.sh
6	Configure Hadoop	Edit core-site.xml, hdfs-site.xml, mapred-site.xml, yarn-site.xml
7	Format NameNode	hdfs namenode -format
8	Start HDFS	start-dfs.sh
9	Start YARN	start-yarn.sh
10	Verify	jps
11	Create HDFS input	hdfs dfs -mkdir -p /input
12	Upload file	hdfs dfs -put file.txt /input
13	Run WordCount	hadoop jar ... wordcount /input /output
14	View result	hdfs dfs -cat /output/part-r-00000

Expected Final Check

✓ Java command works: `java -version`

✓ Hadoop command works: `hadoop version`

✓ `jps` shows NameNode, DataNode, SecondaryNameNode, ResourceManager and NodeManager

✓ `hdfs dfs -ls /input` shows the uploaded input file

✓ WordCount job completes without an output-directory error

✓ `hdfs dfs -cat /output/part-r-00000` displays word counts

Result

Hadoop is installed and configured in Ubuntu, HDFS and YARN services are started, and a sample MapReduce WordCount program is executed successfully using an input file stored in HDFS.

Common Errors During Practice

Problem	What to check
JAVA_HOME is not set	Check the Java path in hadoop-env.sh. It must match the JDK actually installed.
hadoop: command not found	Check ~/.bashrc HADOOP_HOME/PATH lines, then run source ~/.bashrc.
Output directory already exists	Run: hdfs dfs -rm -r -f /output, then run WordCount again.
jps misses HDFS/YARN daemons	Review the XML files and terminal startup errors; also check localhost SSH if start-dfs cannot connect.
Permission denied for /usr/local/hadoop	Make sure the Hadoop data folders exist and are owned by your user.